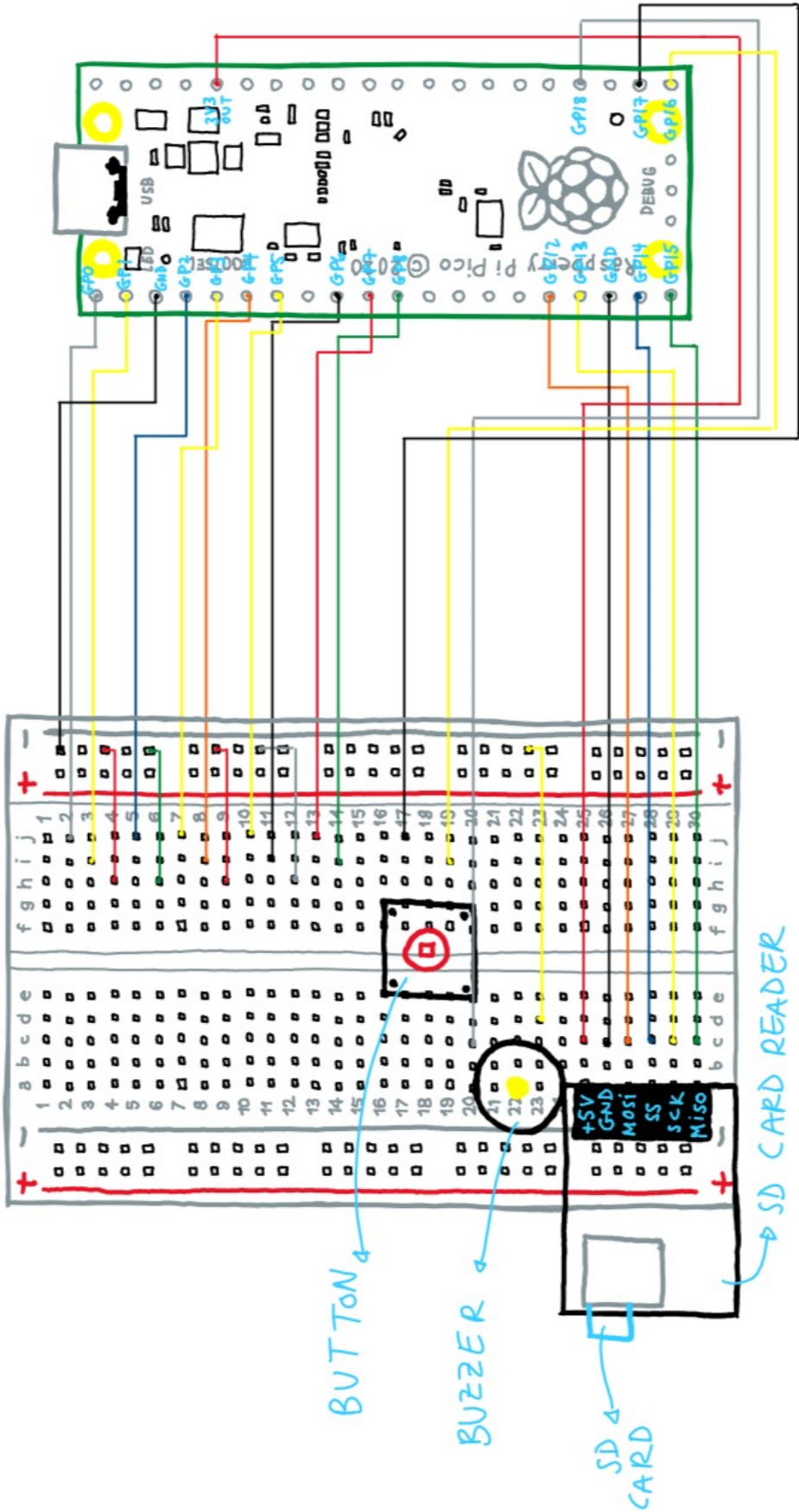


Wind Tunnel Test Rig User Manual

10/05/24

1 Circuit and Pins



The previous figure was a schematic drawing of our fully wired circuit. The pins that are used on the Pico are labelled in blue. The white wires were shown in grey in the schematic. The following list states each pin, the Pico pin it's connected to and the colour of each wire used (colour-coded):

- Pin 1 - GND (RPM ground wire)
- Pin 2 - GP0 (RPM power in)
- Pin 3 - GP1 (RPM signal out)
- Pin 4 - GND (Voltage ground)
- Pin 5 - GP2 (Voltage signal out)
- Pin 6 - GND (Current ground)
- Pin 7 - GP3 (Current power in)
- Pin 8 - GP4 (Current signal out)
- Pin 9 - GND (Load cell ground)
- Pin 10 - GP5 (Load cell serial Tx)
- Pin 11 - GP6 (Load cell serial Rx)
- Pin 12 - GND (ESC ground)
- Pin 13 - GP7 (ESC power out)
- Pin 14 - GP8 (ESC PWM in)
- Button pin 1 - GP17 (button output)
- Button pin 2 - GP16 (button input)
- Buzzer pin 1 - GP18 (buzzer output)
- Buzzer pin 2 - GND (buzzer ground)
- SD pin 1 - 3V3 OUT (+5V)
- SD pin 2 - GND (ground)
- SD pin 3 - GP12 (MOSI)
- SD pin 4 - GP14 (SS)
- SD pin 5 - GP13 (SCK)
- SD pin 6 - GP15 (MISO)

2 Code Description

2.1 Constants

The code after this defines all the constants needed in the main code. These are the minimum and maximum voltage, current, RPM (also converted to revolutions per microseconds), bit depth; the current, voltage and RPM sensitivities and biases; the bit-count, the known PWM frequency for the motor control signal & the duty cycle.

2.2 Functions

The code starts with importing multiple libraries and then moves onto defining all the functions to be used in the main code. These functions are:

- **'measure_motor_voltage()'**: The function's input is the binary output signal from the voltage sensor, and it returns the input voltage to the motor. The function reads this binary value and then using the formula $V_{pico,out} = \frac{V_{ref}}{2^n - 1} \times counts$, calculates the corresponding sensor output voltage read by the Pico. The sensitivity and bias are the slope and constant (respectively) of the linear equation obtained from plotting the voltage values given in the Voltage Sensor Characterisation table in the coursework briefing document. The input motor voltage measured by the voltage sensor is then obtained by using the formula $V_{motor,in} = V_{out} \times Sensitivity$.
- **'measure_motor_current()'**: The function's input is the binary output signal from the current sensor, and it returns the input current to the motor. Just like the previous function, it reads the binary value and follows the same process to get the input motor voltage corresponding to the current passing through the current sensor. However, the sensitivity and bias are given and the formula is a bit different. Linear mapping is then used to obtain the actual input motor current: $LinearMappingOutput = \left[(Input - Ref_{min}) \frac{(S_{max} - S_{min})}{(Ref_{max} - Ref_{min})} \right] + S_{min}$
- **'measure_motor_RPM()'**: The function's input is the binary output signal from the RPM sensor, and it returns the input RPM to the motor. Just like the previous functions, it reads the binary value and follows the same process to get the corresponding voltage outputted by the RPM sensor. Same as in the previous function, sensitivity, bias and linear mapping are used to get the actual RPM.
- **'decode_UARTmsg()'**: The function's input is the serial output Tx line, and it returns the thrust and torque values as an array. Using 'split' it splits the array obtained from the load cell into the id, thrust and torque. The thrust and torque are then stored into an array, 'thrustANDtorque'. Try and except is used so if it can't read it, it just outputs [0,0].
- **'measure_thrust_torque'**: The function's purpose is to check if the load cell is outputting any data. It's input is the serial output Tx line, and it returns the thrust and torque array. It uses an if not-else-if statement to separate the thrust and torque from the array. If there is no serial output to be read, it will simply output the thrust and torque as 0 and the next if statement will then break the code if this condition is true, else it calls the 'decode_UARTmsg' and it returns the thrust and torque.
- **'calculate_ESC_output'**: This function simply reads the analogue value from the ESC and uses linear mapping to get the corresponding RPM value.
- **'change_rpm'**: This function's input is the RPM value ('ESC_output'). It uses an if else statement; if the ESC_output is 0, it prints a statement indicating that throttle is off and instructing the user to turn it on. The else statement uses another if statement within an infinite while loop. The timer is started before the

loop begins and then while the timer is ahead of the sum of the time value and the inverse of the motor control PWM signal frequency, the set outputted duty cycle is calculated from the RPM and then becomes the input of the ESC itself.

- **‘collect_measurements’:** This function’s inputs are the outputs from the voltage, current and RPM sensor & the Load Cell. It calls the `measure_motor_voltage`, `measure_motor_current`, `measure_motor_RPM` and `measure_thrust_torque` and records all these values in separate variables (the thrust and torque are called from the array and also stored as separate functions). It then returns all these values, arranged in an array.
- **‘calculate_averages’:** This function’s inputs are also the outputs from the voltage, current and RPM sensor & the Load Cell. It establishes initial empty arrays for the voltage, current, RPM, thrust and torque values as well as zero variables for their sums. It starts a timer and then while the timer is under 5 seconds (requirement 2 d) it calls the ‘collect_measurements’ function and starts storing all the values, adding these values one by one in the previously defined empty arrays and zero variables (there is a counter ‘i’ defined before the loop starts and a 1 is added to it after every iteration of the loop). The averages are then calculated by dividing the sums by the length of the now not empty arrays. These averages are then recorded and outputted as an array.

2.3 Main Code

We first create two linspace arrays for the throttle analogue and RPM settings & an array for the windspeeds (requirement 2 c).

The rest of the code exists in a for loop that runs for every value in the windspeed array after discarding the values recorded over 2 seconds every time the throttle setting changes. Inside it is an if statement to check the button setting; if off, the code breaks. If the button is on, then the code goes into a second for loop that runs for every element in the throttle analogue settings array.

The second for loop initially stops the code for every change in throttle setting for 2 seconds (requirement 2 e). The ‘calculate_ESC_output’ and ‘change_RPM’ function are then called to set the RPM to the new throttle setting as per the loop iteration. It also calls the ‘collect_measurements’ function to collect all the values as an array. Inside the for loop is an if statement after a timer is setup that outputs the averages of voltage, current, rpm, thrust and torque values over the last 5 seconds (the condition for the if statement ensures this by dividing the timer value at that moment and seeing if its remainder is 0).

Then come the serial monitoring messages that are outputted every second using ‘time.sleep’ (at 1 Hz) (requirements 4 a to c). The voltage, current, RPM, thrust and torque values are simply taken from the array outputted by the previously called ‘collect_measurements’ function. The power however is first calculated from the voltage and current and then outputted. There is also a message to output the averages.

After this is the code for voltage and current warning (requirement (4 d and e). First, we establish an empty array to store the current values for the warning system and start a timer using ‘time.monotonic’. There is a simple if-else statement that activates the buzzer for half a second if the motor input voltage drops below 22 V. Then there is the if-else statement inside an if-elif statement, and all of this is within an infinite while loop. The while loop starts storing the current values (from the ‘collect_measurements’ function) for this part in the previously defined array. Then the if-elif statement checks if the current value is above the threshold of 20 A or not. If it isn’t, it will simply stop the timer and the loop will start again. If not, then the code falls into an if-else statement that activates the buzzer for half a second if it’s been longer than 20 seconds and the current has been above 20 A for that time, otherwise it simply starts the timer again.

The last part of the code outputs the data into a csv file by compiling all the collected data into a table. This table is initialised with empty columns and column header lines for all outputs from the functions. Then, as the data is collected, a while loop stores it in the columns of the initialised table. After each run of the loop, a new row of data is appended to the bottom of the table. Once the testing is fully complete and all the data has been recorded in the table, the code is saved into a csv file with a unique name. All of this is done using the ‘pandas’ library.

Finally, the code comes out of the while loop, turns the switch back off and breaks the for loop, ending the code’s run.

3 User Instructions

The user will first have to connect the wind tunnel test rig (all 14 pins) into the corresponding pin sockets in the breadboard and then press run. If the button is off, the user will receive the “Switch button to on” message. Once the switch has been turned on, the code will automatically start doing its work until the time to save the data into the csv file comes. At this moment, the user will be asked to give a name to the csv file, ensuring they include the .csv extension. If during the code’s run, the voltage falls below 22 V, or if the current stays above 20 A for more than 10 seconds, then the user will be shown a warning message indicating this. During this time, the user will see serial monitoring messages with the windspeed, throttle setting, thrust, torque, voltage, current, RPM and power values & averages over the previous 5 seconds of these values every second. The code will then switch the system off and wait to be restarted.